M.TECH CYBER SECURITY — MID-SEMESTER DISSERTATION EVALUATION

AI-Assisted Detection of LOLBin-Based Privilege Escalation

Phase 1 of 2: Generic Behavioral Detection Foundation

PRESENTER

Anvesh Dhiraj Shivhare
M.Tech Cyber Security, Semester III

INSTITUTION

School of Cyber Security & Digital Forensics
National Forensic Sciences University, Gandhinagar

RESEARCH TRACK

Defensive EDR / Host Behavioral Anomaly Detection
Unsupervised Learning + Zero-Shot Explainability

SUBMISSION CONTEXT

October 2026
Academic Year 2026–2027 (Mid-Semester Panel)

◆ THREAT MODEL & STRATEGIC PIVOT

Problem Formulation: The Living-off-the-Land Threat

Why legitimate dual-use administrative binaries evade signature-based AV and static EDR

1. Adversarial Abuse of Dual-Use Utilities (LOLBins / GTFOBins)

Modern threat actors avoid compiling custom malware binaries. Instead, they weaponize pre-installed, digitally signed system utilities (e.g., powershell.exe, certutil.exe, find, awk, tar, sudo) for post-exploitation and privilege escalation (PrivEsc).

2. Failure of Static Signatures & Crippling Rule Fatigue

Because LOLBins inherit valid OS signatures and run from trusted system paths, traditional signature-based AV is ineffective. Static EDR rules face an irreconcilable tradeoff: either trigger severe False Positive alert fatigue on routine administration or permit intrusions.

3. The Defensive Research Gap (Our Strategic Pivot)

Offensive AI automation (auto-exploitation, LLM payload mutation) is saturated in literature (PentestGPT, hackingBuddyGPT). Detecting subtle dual-use commands via unsupervised behavioral anomaly detection paired with zero-shot LLM explainability is an open, defensible gap.

4. The Sequence Blind Spot (Isolated vs. Chained Execution)

A dual-use utility (e.g., 'whoami' or 'id') looks completely benign in isolation. Malicious intent only manifests across consecutive command transitions within an attack session, requiring sequence-aware behavioral analysis rather than isolated inspection.

◆ LITERATURE TAXONOMY (IEEE / APA CITATIONS)

Theoretical & Literature Foundation

Mapping peer-reviewed foundational literature directly to each architectural design decision

Isolation Forest Algorithm
Anchor: Liu, Ting, Zhou (IEEE ICDM 2008)

Exploits fundamental property: 'anomalies are few and different'. Isolates anomalies via random recursive partitioning without requiring labeled attack training traces.

Why Unsupervised Learning
Anchor: Stamp (2022, arXiv:2208.12836)

Demonstrates severe scarcity of labeled malicious LOLBin data. Proves supervised models overfit to seen syntax and collapse when facing novel post-exploitation techniques.

Anomaly Detector + LLM Pairing
Anchor: Ali & Kostakos (2023, HuntGPT)

Establishes architectural precedent combining statistical ML anomaly detection with zero-shot LLM explainability to translate mathematical deviations into actionable SOC triage.

LLM Grounding & Hallucination Guard
Anchor: Houssel et al. (2024, arXiv:2408.04342)

Defines methodology for strictly grounding LLM reasoning in empirical feature deviations and z-scores, prohibiting ungrounded external assumptions.

Sliding-Window Command Context
Anchor: Springer (2026)

Proves empirically that evaluating a sliding window of consecutive commands significantly outperforms isolated single-command inspection on Linux endpoint telemetry.

Anomaly Weakness on Stealth / Obfuscation
Anchor: Systematic Literature Review (SLR 2026)

Exhaustive review establishing that Isolation Forest excels on overt anomalies but exhibits known degradation on stealthy/obfuscated intrusions—grounding our Tier 3 test.

◆ END-TO-END PIPELINE ARCHITECTURE

System Architecture: Behavioral Detection Pipeline

End-to-end ingestion, feature representation, unsupervised scoring, and grounded LLM attribution

1. Telemetry Ingestion

AuditdTelemetrySource

- Ingests raw Linux auditd streams
- Correlates SYSCALL, EXECVE, PATH, CWD
- Decodes hex proctitles into plaintext
- Extensible adapter architecture

REAL DATA (auditd)
& Synthetic Adapter

→

2. Feature Engineering

TelemetryFeatureExtractor

- Lineage rarity (parent-child co-occurrence)
- Shannon entropy & special char ratio
- Sensitive path touch & privilege flags
- Sliding Command Window (N=3)

Real Fitted Statistics
(Zero Retraining)

→

3. Anomaly Detection

Isolation Forest (Unsupervised)

- 150 Isolation Trees (contamination=0.025)
- Trained strictly on benign baseline
- Continuous standardized feature space
- Calibrated decision boundary = 0.50

Real Auditd Baseline
(25,495 Events)

→

4. SOC Explainability

LLM Attribution & Triage

- Zero-shot Tier-3 SOC analyst prompt
- Consumes JSON feature anomaly payload
- Outputs ATT&CK ID & confidence score
- Deterministic offline fallback engine

Architecture Validated
(No Label Leakage)

Data Provenance Distinction: Detection & Feature Pipeline is trained and evaluated on REAL Ubuntu auditd telemetry. Multi-tier adversarial benchmarking (Tiers 1–3) was validated against synthetic lab streams during architectural development.

Implementation Status: Phase 1 Scope & Boundary

Transparent accounting of what is delivered and operational vs. synthetic and Phase 2 targets

BUILT & OPERATIONAL ON REAL DATA

Phase 1 Delivered (Verified on Linux Host)

- Real Linux auditd ingestion parser correlating SYSCALL, EXECVE, PATH, and CWD records into unified event objects.
- Fitted baseline Isolation Forest on 17,846 real events (70% of 25,495 Ubuntu dataset) with persistent joblib model bundle.
- Standalone inference engine (predict.py) scoring fresh endpoint telemetry with ZERO retraining or refitting.
- Real attack script executions captured and evaluated against live VM auditd logs (find, python3, tar, awk GTFOBins).

SYNTHETIC-ONLY (BENCHMARK VALIDATION)

Development Architecture Testing

- Three-tier attack complexity matrix (Tier 1 overt, Tier 2 chained, Tier 3 obfuscated) evaluated on synthetic dataset.
- Supervised vs. unsupervised generalization experiment reproducing Stamp (2022) with SMOTE comparators.
- Sliding-window size ablation study across N=3, 4, 5 on synthetic multi-step sequences.
- Zero-shot LLM attribution tested across synthetic scenario payloads.

PHASE 2 ROADMAP (DISSERTATION TA2)

Future Work & Engineering Upgrades

- LOLBin argument semantic embeddings & AST-level parsing to eliminate blind spots on subtle syntax (e.g., awk).
- Sequence transition-gradient features (binary_changed, entropy_delta) to detect low-signal reconnaissance chains.
- Real-data capture for multi-step chained and obfuscated attacks executed directly inside isolated lab VMs.
- Cross-OS Windows Sysmon adapter integration (EventID 1 process creation, EventID 11 file creation).

● REAL TELEMETRY: LINUX AUDITD (UBUNTU BASELINE)

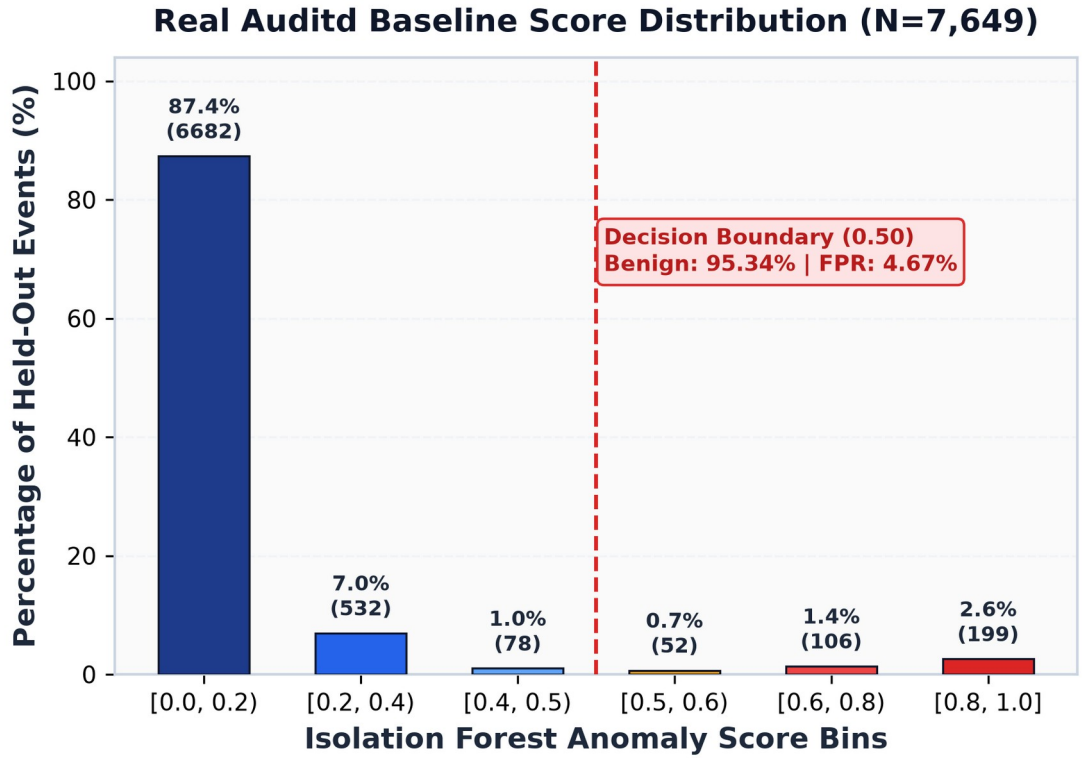
Real Data Result: Baseline Validation & FPR Calibration

Unsupervised Isolation Forest evaluation on 25,495 captured Linux auditd telemetry events

Held-Out Baseline Score Distribution (N = 7,649)

- **Total Telemetry Captured:** 25,495 correlated auditd events (Ubuntu)
- **Experimental Split:** 70% Train (17,846 ev) / 30% Held-Out Test (7,649 ev)
- **Empirical FPR on Held-Out:** 4.67% (357 false positives / 7,649 events)
- **Median Score (50th %ile):** 0.0337 (Deep interior of benign hyperspace)
- **90th / 95th Percentiles:** 0.2618 / 0.4564 (Safely below 0.50 threshold)
- **Strongly Benign Bins (<0.2):** 87.36% (6,682 events) score below 0.20
- **Sub-Threshold Normal (<0.5):** 95.34% of benign telemetry falls below cutoff
- **Sliding Window Advantage:** FPR drops from 7.57% (isolated) to 4.67% (window)

Per-Binary Baseline False Positive Discrimination:
- **find: 0.0% FPR (0/66)** | **awk: 0.0% FPR (0/381)**
- **tar: 4.17% FPR (1/24)** | **python3: 25.39% FPR (49/193)**



● REAL TELEMETRY: LIVE GTFOBIN EXECUTION AUDITD

Real Data Result: Live GTFOBin Detection & Context Dynamics

Standalone inference (predict.py) flags real GTFOBins and captures sliding-window elevation

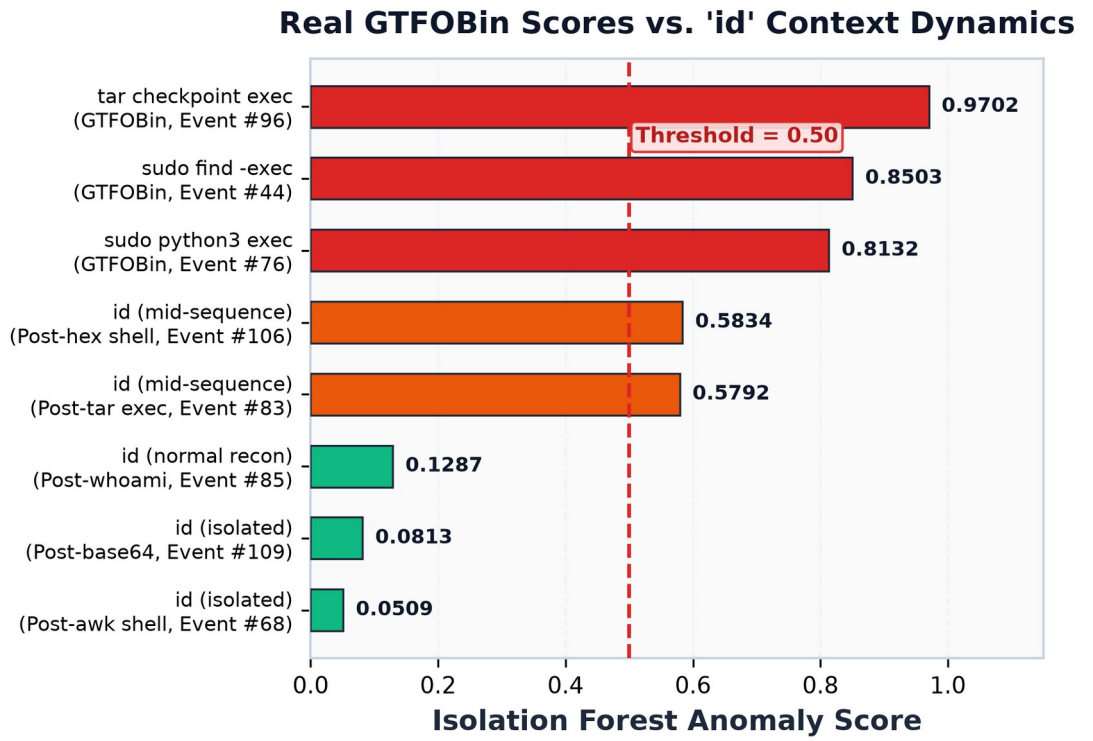
Inference on Live Attack Log (data/live-attack-benign.log)

Overt GTFOBin Invocations Successfully Detected:

- sudo find . -exec /bin/sh... ➔Anomaly Score: 0.7424 – 0.8503 (FLAGGED)
- sudo python3 -c import os... ➔Anomaly Score: 0.7007 – 0.8132 (FLAGGED)
- tar --checkpoint-action=exec... ➔Anomaly Score: 0.8036 – 0.9702 (FLAGGED)

The 'id' Command Context Experiment (Springer 2026 Validation):

- Normal / Isolated Context: 'id' executed during benign baseline scores 0.0509 – 0.1287 (Events #68, #80, #85, #109 — Sub-threshold Normal).
- Mid-Attack Sequence: Exactly the same 'id' command executed immediately following tar checkpoint or hex shell jumps to 0.5792 (#83) and 0.5834 (#106) — CROSSING threshold!
- Concrete Proof: The sliding window correctly captures multi-step behavioral elevation rather than isolated command text.



■ HONEST SCIENTIFIC LIMITATION (REAL AUDITD FINDING)

Honest Limitation 1: Undetected GTFOBin (awk)

The awk-based privilege escalation technique scored strictly below 0.50 and evaded detection

Concrete Empirical Finding: sudo awk BEGIN {system(...)} Was Not Detected

During live attack simulation (live-attack-benign.log, Events #57–#66), the awk-based shell spawn technique produced scores between 0.0617 and 0.4928 across all attempts. Every single invocation remained sub-threshold (< 0.50 decision boundary), representing a complete false negative.

1. Benign Distribution Masking

- awk appears 381 times in the held-out benign baseline.
- The model established a strong prior for awk as routine administrative text processing.
- Table 2C shows awk has 0.0000 FPR on benign data; the detector heavily normalizes awk activity.

2. Feature Space Insensitivity

- Syntax 'BEGIN {system(...)}' does not contain explicit exploit tokens like '-exec', 'checkpoint', or 'bypass'.
- Character Shannon entropy remains moderate (~3.2–3.6), indistinguishable from complex sed/awk queries.
- Token hit counter recorded 0 hits.

3. Privilege Signal Latency

- auditd captured the awk process execution under sudo, but the child shell was spawned internally.
- The instantaneous command did not trigger a logged UID change at the primary audit event boundary.
- Generic lineage features failed to detect execution pivot.

Key Academic Takeaway: Generic behavioral features struggle with syntax-subtle LOLBins. Catching awk requires semantic argument modeling and token n-gram embeddings (Phase 2), not just generic command length and entropy.

■ HONEST SCIENTIFIC LIMITATION (SEQUENCE FEATURE GAP)

Honest Limitation 2: Low-Signal Reconnaissance Blind Spot

Multi-step discovery chains evade detection when every command in the chain is individually low-signal

Concrete Empirical Finding: whoami →id →hostname →uname -a Chain Was Missed

In both baseline logs and live simulation, a sequence composed purely of standard discovery utilities (whoami, id, hostname, uname -a, groups, ls, pwd) produced anomaly scores strictly between 0.01 and 0.13. The sliding window failed to trigger an alert despite coherent reconnaissance activity.

Order-Invariant Feature Aggregation

• Current sliding window computes scalar aggregates: max, avg, and sum.
• It lacks transition dynamics (e.g., priv_transition_delta, binary_changed, entropy_gradient).
• The window treats [whoami, id, uname] identically to repeated benign commands.

Dependence on an Overt 'Anchor' Step

• Synthetic Tier 2 chains reached 100% chain interception ONLY because each chain contained at least one overt high-signal step (e.g., sudo tar checkpoint).
• The overt step triggered the window; the window did not detect the subtle reconnaissance steps.

Lack of Sequence Transition Modeling

• Isolation Forest partitions vector coordinates; it does not model Markov state transitions or temporal progression.
• When all steps have 0 token hits and no privilege change, window cumulative sums remain safely below the 0.50 threshold.

Key Academic Takeaway: Unsupervised Isolation Forest cannot infer malicious recon intent from zero-signal commands without explicit sequence transition features (binary_changed, token_acceleration), which form our primary Phase 2 objective.

▲ SYNTHETIC DATA: ARCHITECTURE VALIDATION BENCHMARK

Synthetic Validation: Multi-Tier Sensitivity & Overfitting

Controlled architectural experiments conducted during development to validate theoretical models

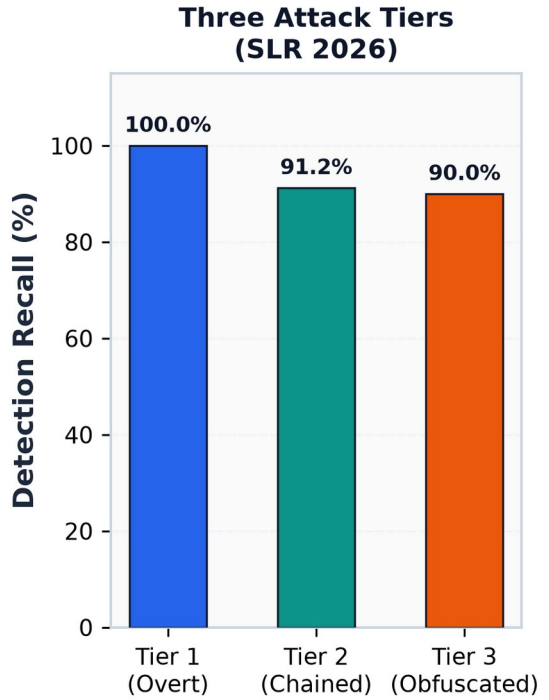
Controlled Lab Experiments (Synthetic Sysmon + Auditd)

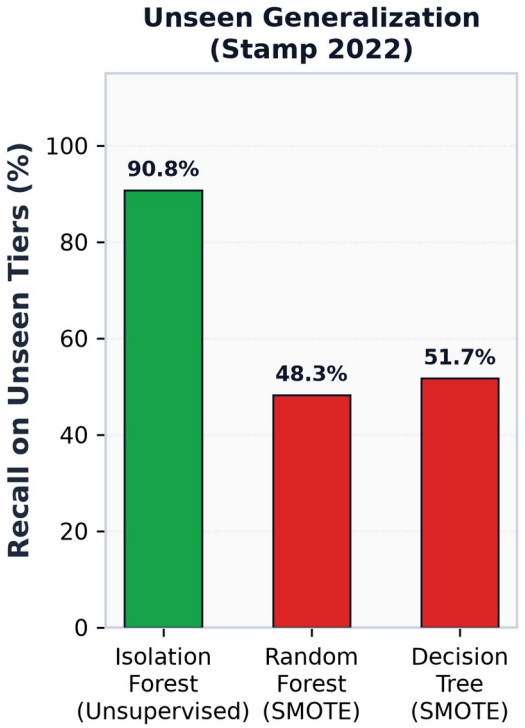
Three Attack Complexity Tiers (SLR 2026 Validation):

- Tier 1 (Overt / Isolated): Recall 100.0%, Precision 0.9901, F1 0.9950 (300 events)
- Tier 2 (Chained Multi-Step): Recall 91.23%, Precision 0.9738, F1 0.9420 (285 events)
- Tier 3 (Obfuscated / Stealthy): Recall 90.00%, Precision 0.9507, F1 0.9247 (150 events)
- Empirical confirmation: Monotonic degradation matches SLR 2026 theorem on obfuscation limits.

Supervised Generalization Failure (Reproducing Stamp 2022):

- Isolation Forest (Unsupervised): 90.80% Recall on unseen Tiers 2 & 3 with ZERO attack labels in training.
- Random Forest (Supervised + SMOTE): Achieves 100% on seen Tier 1, but collapses to 48.28% Recall on unseen Tiers 2 & 3.
- Decision Tree (Supervised + SMOTE): Collapses to 51.72% Recall on unseen Tiers 2 & 3.
- Scientific Validation: Proves supervised classifiers heavily overfit to seen syntax, validating our unsupervised design.





◆ ENGINEERING STACK & LAB ENVIRONMENT

Tools, Technology & Lab Infrastructure

Modular, reproducible engineering stack utilizing standard open-source systems and ML frameworks

Lab Virtualization & Telemetry

Isolated VM Simulation Environment

- Hypervisor: KVM / QEMU managed via virt-manager on Linux.
- Target VM: Ubuntu Server 24.04 LTS (isolated, non-networked).
- Telemetry Sensor: Linux auditd capturing SYSCALL, EXECVE, PATH, CWD.
- Safe Artifacts: bash and powershell simulation scripts decoupled from host.

Machine Learning & Pipeline Core

Unsupervised Anomaly Engine

- Python 3.14 / 3.12 scientific compute environment.
- scikit-learn: IsolationForest (150 estimators), StandardScaler.
- imbalanced-learn: SMOTE used strictly for supervised benchmark comparisons.
- joblib: Standalone pipeline bundle serialization (detector_model.joblib).

Feature Engineering & Windowing

Sequence Behavioral Analytics

- Custom continuous Shannon entropy & character dynamics calculation.
- Sliding Command Window (N=3 to 5) aggregating session context.
- Process lineage rarity mapping (parent-child co-occurrence frequency).
- Dual-OS token dictionaries (Linux GTFOBins & Windows LOLBins).

Explainability & SOC Console

Zero-Shot Cognitive Layer

- LLM API Integration: OpenAI / Anthropic / Groq REST endpoints.
- Strict Zero-Shot Prompt: Constrained to JSON feature anomalies (no hallucination).
- Deterministic Offline SOC Fallback: Full testability without API keys.
- Streamlit Dashboard: Interactive triage console with telemetry toggle.

Research Roadmap: Transition to Phase 2 (TA2)

Evolving from generic behavioral anomaly detection to LOLBin-specific syntactic modeling

1. LOLBin Argument Semantic Modeling

Objective: Eliminate blind spots like awk — Transition from generic command entropy to semantic token embeddings (Ongun et al. 2021) and AST argument parsing. Distinguish routine text parsing from subshell invocations (e.g., BEGIN {system()}).

2. Sequence Transition-Gradient Features

Objective: Detect low-signal discovery chains — Augment the sliding window with dynamic inter-command features: binary_changed flag, token_acceleration, and session entropy gradients to catch reconnaissance chains before privilege escalation occurs.

3. Multi-Tier Real VM Telemetry Capture

Objective: Replace synthetic benchmark with real VM traces — Execute full Tier 2 (chained sequences) and Tier 3 (obfuscated GTFOBins) attack matrices inside isolated Ubuntu VMs, capturing 100% real auditd logs to validate production generalization.

4. Cross-OS Windows Sysmon Integration

Objective: Dual-OS EDR coverage — Implement a dedicated Windows Sysmon XML/JSON parser (EventID 1 process creation, EventID 11 file creation) and benchmark prevalent Windows LOLBins (certutil, powershell, mshta, rundll32).